

ALGORITHM ARENA PLATFORM

Execution and Sandboxing

Security, Resource Isolation, and Runtime Environment

DOCUMENT C: PLATFORM-WIDE INFRASTRUCTURE SPECIFICATION

Document Level:	Document C (Global Platform Standard)
Scope:	Universal across all Algorithm Arena challenges
Security Model:	Zero-Trust Untrusted Code Execution
Version:	1.0.0
Date:	August 2026

This specification defines the universal runtime isolation, hardware resource limits, security sandbox constraints, inter-process communication (IPC), and multi-language toolchains powering Algorithm Arena.

Contents

1	Overview and Security Principles	1
1.1	Core Principles	1
2	Resource Limits and Quotas	1
3	Sandbox Isolation Architecture	1
3.1	Linux Kernel Namespaces	1
3.2	Seccomp-BPF Syscall Filtering	2
4	Multi-Language Runtime Toolchains	2
4.1	Python 3 Runtime	2
4.2	C++ Runtime	2
4.3	Rust Runtime	2
4.4	Java Runtime	2
4.5	Go Runtime	2
5	Inter-Process Communication (IPC) Protocol	3

1 Overview and Security Principles

Algorithm Arena executes arbitrary, user-submitted source code and compiled binaries from untrusted external participants. To ensure competitive fairness, benchmark integrity, and host system security, the platform operates under a strict **Zero-Trust Sandbox Model**.

1.1 Core Principles

1. **Strict Hermetic Isolation:** Untrusted code cannot access network interfaces, host filesystem structures, hardware devices, or sibling container memory.
2. **Absolute Determinism:** Hardware clocks, thread scheduling, and PRNG seeds are virtualized or constrained to produce identical execution trajectories on identical inputs.
3. **Hard Multi-Tier Resource Limits:** Exact CPU, memory, time, and file descriptor bounds are enforced at the Linux kernel level via cgroups v2 and seccomp-bpf.

2 Resource Limits and Quotas

Resource	Standard Limit	Enforcement Mechanism
CPU Allocation	1 dedicated vCPU	<code>cpuset.cpus</code> core pinning
Memory (RAM)	≤ 256 MB	<code>memory.max</code> (cgroups v2 OOM-killer)
Per-Step Time	≤ 10 ms	Real-time POSIX monotonic timer watchdog
Per-Match Time	≤ 2.0 seconds total	Aggregate wall-clock timeout
Network Access	Strictly Disabled	Network namespace with unconfigured loopback
Disk Storage	Read-only root + 16MB tmpfs	Ephemeral non-persistent RAM disk
Open File Descriptors	≤ 16	<code>RLIMIT_NOFILE</code>
Thread / Process Count	≤ 1 process / thread	<code>pids.max = 1</code> (No forks/multithreading)

3 Sandbox Isolation Architecture

3.1 Linux Kernel Namespaces

Each submission instance is encapsulated within six isolated kernel namespaces:

- **Mount (`CLONE_NEWNS`):** Mounts a minimal, read-only rootfs containing only language runtime essentials.
- **PID (`CLONE_NEWPID`):** Isolates the process table; the participant controller is PID 1.
- **Network (`CLONE_NEWNET`):** Disconnects all physical and virtual network interfaces.
- **IPC (`CLONE_NEWIPC`):** Blocks host shared memory segments and semaphores.
- **UTS (`CLONE_NEWUTS`):** Isolates host system hostname and domain information.
- **User (`CLONE_NEWUSER`):** Maps container root to an unprivileged host UID.

3.2 Seccomp-BPF Syscall Filtering

A strict Berkeley Packet Filter (BPF) intercepts all system calls executed by user code:

- **Allowed Syscalls:** read, write (standard pipes only), futex, mmap, brk, clock_gettime, exit_group.
- **Blocked Syscalls (Instant Termination):** fork, vfork, clone, execve, socket, connect, bind, ptrace, kill, chroot, open (write-mode).

4 Multi-Language Runtime Toolchains

The execution platform provides standardized language toolchains:

4.1 Python 3 Runtime

- **Interpreter:** CPython 3.10+ (64-bit Linux).
- **Pre-installed Numerical Packages:** numpy, scipy.
- **Prohibited Standard Modules:** os, sys, subprocess, socket, ctypes, threading, multiprocessing (blocked via import hooks and seccomp filters).

4.2 C++ Runtime

- **Compiler:** GCC 13+ / Clang 17+ with flags:

```
1 g++ -O3 -std=c++20 -static -fno-rtti -Wall -Wextra -s submission.cpp -o controller
2
```

- **Standard Libraries:** Full ISO C++20 standard library (STL).

4.3 Rust Runtime

- **Toolchain:** Rust Stable (1.75+).
- **Compilation Command:** cargo build -release -target x86_64-unknown-linux-musl.

4.4 Java Runtime

- **JVM / JDK:** OpenJDK 21 LTS (64-bit).
- **Execution Flags:**

```
1 java -XX:+UseSerialGC -Xms32m -Xmx220m -Dfile.encoding=UTF-8 -jar submission.jar
2
```

- **Security Policy:** Custom Java SecurityManager / JVM agent blocking unauthorized reflective access and network sockets.

4.5 Go Runtime

- **Compiler:** Go 1.22+ (gc toolchain).
- **Compilation Command:**

```
1 CGO_ENABLED=0 go build -ldflags="-s -w" -o controller main.go
2
```

- **Characteristics:** Statically linked pure binary with minimal memory footprint.

5 Inter-Process Communication (IPC) Protocol

The platform executes matches via a **Driver-Worker Architecture**:

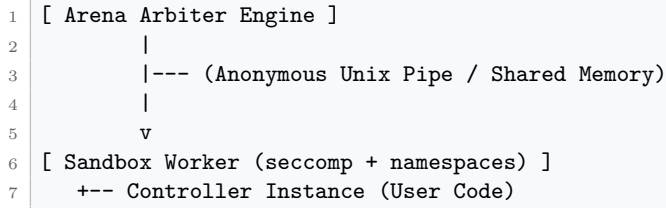


Figure 1: Arena Arbiter and Sandbox Worker communication channel.

1. **Handshake:** Arbiter sends RESET(seed=0x3F8A) over stdin. Controller responds with ACK.
2. **Step Dispatch:** Arbiter transmits JSON/binary observation packet.
3. **Action Reply:** Controller writes integer action token within 10ms.
4. **Watchdog Enforcement:** If timeout expires, Arbiter forcefully SIGKILLS the sandbox worker.